μKV: Energy- and Thermal-Aware KV-Cache Eviction for LLM Inference on a Phone

Md Romyull Islam
Kennesaw State University
Marietta, Georgia, USA
romyullislam2012@gmail.com

ABSTRACT

On a phone the KV cache, not compute, limits sustained LLM decode. The policies that bound it were built for server GPUs. Three properties of a real on-device engine break them. One cell array serves every head and layer, so a cell is freed only when every head and every layer drops it. SnapKV reports an 8.2 times better memory efficiency; on the phone GPU it retains 68% of the cells at a matched budget and 94% at its own. Reading attention scores forces the FlashAttention-off path, so on the phone GPU every score-reading evictor decodes at 0.12 to 0.20 times the full cache on Llama-3.2-1B. Compaction through the state API needs a second full context, and the OS kills Phi-3-mini at 16K. μKV respects all three. It scores tokens inside the FlashAttention prefill graph through a small side node. It selects one keep set for all heads and layers. It compacts in place, so peak memory never rises. Two controllers sit beside the cache. A reduce-only clock watchdog glides the CPU and GPU clocks down before the vendor acts, and on the CPU it keeps the vendor's throttle unreachable. A per-request scheduler picks the GPU clock plan and the answer cap from the battery state, and two loops correct it. On a OnePlus 15, μKV decodes 4.8 times faster than the full cache on the CPU, at 7% of the cells and 62% less energy, and 1.23 times faster on the Adreno GPU. It ties the full cache on 392 retrieval cells and stays within 0.1 to 3.7 F1 on LongBench at 13% of the cache. With 1-bit weights it finishes a workload that deep-throttles the full cache. Through a real discharge from 91 to 11%, its two lower tiers cut energy per request by 15% and 43%.

KEYWORDS

Mobile LLM inference, KV cache eviction, FlashAttention, thermal management, energy-aware scheduling, on-device AI

1 INTRODUCTION

Small instruction-tuned models now run interactively on flagship phones. For a sustained generation the wall is the KV cache, not the matrix multiply. Every decode step reads the whole cache once per layer. Past a thousand tokens the step is bound by DRAM bandwidth. The DRAM heats, the vendor thermal engine cuts the clock, and the battery drains. Cache size sets traffic, traffic sets power, power sets temperature, temperature sets the throttle, and the throttle sets throughput. A mobile cache policy must therefore be judged on quality, throughput, heat and energy together, on real hardware.

The policies that bound the cache were built for server GPUs and judged by perplexity alone: H2O [1], TOVA [3], SnapKV [4], StreamingLLM [2] and Ada-KV [6]. We ran each in its published configuration on a OnePlus 15 inside `llama.cpp`. Three properties of the engine broke them.

- **Budgets do not materialize.** The engine keeps one cell array shared by every head and layer. A cell is freed only when every head and every layer drops it. SnapKV selects per head and reports an 8.2 times better memory efficiency at 16K inputs [4]. At a matched budget of 1024 per head on the phone GPU it keeps 68% of the cells on Llama-3.2-1B and 97% on Phi-3-mini. At its own budget of 2048 per head it keeps 94%. H2O reports a 5 to 10 times smaller cache at a 20% budget [1]; at that budget here it keeps 92 to 100%.
- **Scores cost the fast path.** A policy that reads post-softmax attention cannot use the fused FlashAttention kernel. On Llama-3.2-1B every score-reading evictor decodes at 0.12 to 0.20 times the full cache on the phone GPU, so evicting makes it slower.
- **Compaction doubles memory.** The state API restores survivors into a second context. Phi-3-mini at 16K needs two 6.4 GB caches plus 2.4 GB of weights, and the OS kills it. The same call fails on a 24 GB desktop GPU.

μKV is an evictor designed around these facts, with the two controllers a phone needs beside it (Figure 1). It computes selection scores inside the FlashAttention prefill graph through a small side node. It selects one keep set of K cells for all heads and layers, so eviction frees cells. It compacts in place: survivors slide into a dense prefix inside the tensors prefill already allocated. A reduce-only watchdog steps the CPU and GPU clocks down along temperature ladders anchored at the measured vendor triggers. A per-request scheduler picks the GPU clock plan and the answer cap from the battery state. It predicts the request's cost from a measured table, and two loops correct its lever when time or energy misses.

We evaluate on a OnePlus 15 (Snapdragon 8 Elite Gen 5, Adreno 840) with four models and nine policies. Quality is perplexity on a slice disjoint from the prompt, retrieval on 392 needle cells, and LongBench F1. Cost is prefill and decode time, live cells, peak resident set, DDR and skin temperature, and system energy, all under a cooling gate. This paper makes five contributions.

(i) The realizability gap: nominal per-head budgets do not compact on a sequence-level engine, measured for five policies on four models (§6.1, §6.2).
(ii) In-graph scoring, one keep set and in-place compaction, which keep an attention-driven evictor on the FlashAttention path and make an infeasible configuration run (§3).
(iii) A thermal watchdog with CPU and GPU ladders anchored at the measured vendor trigger, and the negative result that cache size does not cap peak temperature (§4, §6.4).

(iv) An energy-aware scheduler with one lever and two loops, measured through a real battery discharge and under provoked disturbances (§6.5).
(v) A multi-axis on-device evaluation with every baseline in its own published configuration (§6).

2 BACKGROUND AND MOTIVATION

Decode is bandwidth-bound. Autoregressive decode reads all cached keys and values every step. Phi-3-mini holds 0.375 MiB of f16 KV per token, so a 2K context is 768 MiB, and decode reads all of it once per layer per step. On LPDDR5X at about 50 GB/s this dominates the step. The cache also sets heat. Across our 372-cell on-device dataset, live cache correlates with DDR temperature at r=0.80 and with throughput at r=−0.77.

Eviction policies. H2O [1] keeps heavy hitters by accumulated attention plus recent tokens, split evenly, and reads attention every step; it reports a 5 to 10 times smaller cache at a 20% budget. TOVA [3] drops the position with the least attention from the last query, averaged over the heads of a layer, and reports results nearly on par with the full model at, in some cases, one eighth of the cache, with 4.8 times the throughput. SnapKV [4] freezes a per-head top-K at the end of prefill from an observation window and reports 3.6 times faster decode and an 8.2 times better memory efficiency at 16K inputs. StreamingLLM [2] keeps 4 sink tokens and a sliding window with no scores; it reports 22.2 times the speed of window recomputation and states that it adds no long-term memory. Ada-KV [6] allocates one layer budget across heads from a global top-B with a safeguard of 0.2, and needs FlashAttention-2's variable-length kernel [11] plus a custom CUDA kernel, both absent on the mobile stack. KIVI [5] quantizes instead of evicting. KVzip [15] scores by reconstruction. Both are orthogonal.

The architectural conflict. FlashAttention [10] fuses softmax into the kernel and never writes the attention tensor to memory. A policy that reads attention scores cannot use the fused kernel and is locked to the slow path. On the phone this, not retention, decides speed. Two policies can retain the same few cells and differ fivefold, and which kernel each leaves behind is what separates them. The distinction matters for who can be fixed: a policy that reads no scores can be put back on the fused kernel, while a per-head evictor cannot, because reading scores is how it selects.

The engine. `llama.cpp` [14], like every on-device engine we know of, stores the cache as one array of cells indexed by position and shared across heads and layers. Its cache API removes positions, so a per-head keep set can only be realized as the union over heads. Compaction goes through the state API, which serializes the cache and restores it into a second context. Neither fact is visible on a server with per-head paged storage. Both decide what a policy can do on a phone.

Thermal and energy control on phones. zTT [20] and FUSE [21] act on CPU and GPU frequency from temperature or phase, with no cache signal. The vendor engine on our SoC deep-throttles the prime cores to 883 MHz the moment the battery reaches 50 ℃. It caps the GPU at 726 MHz when DDR passes 64 ℃. No shipped phone changes an LLM's clock, context or answer length by battery state. EnerInfer [27] is the closest research system and excludes battery level by design. Kim, Imes and Hoffmann [26] prove that an energy-optimal schedule under a deadline uses at most two configurations. JouleGuard [25] couples an accuracy loop and an energy loop through model error. μKV's scheduler applies both results.

3 μKV DESIGN

Figure 1 shows the system. The inference pass is unmodified `llama.cpp` with three insertions between prefill and decode: score, select, compact. Figure 2 shows where the scores come from, how one keep set is chosen, and why it compacts while per-head keep sets cannot.

3.1 Scoring inside the FlashAttention graph

The selection below needs post-softmax attention, which the fused kernel never materializes. μKV adds a small `kq_evict` side node to the prefill graph instead of leaving the fast path. The node computes softmax($KQ^\top$) for the last 16 queries of the last prefill chunk only, since earlier chunks' scores are overwritten anyway. It reduces over queries to one score per layer, head and position, $\bar{A}_{l,h,p}$. The node is a graph output, so the mid-graph interception that corrupts this Vulkan backend cannot touch it. It costs 1.2% of prefill on the CPU (paired, n=29) and 138 against 131 s on the GPU. Selection is bit-identical to a FlashAttention-off capture, verified by teacher-forced perplexity.

3.2 One keep set for all heads and layers

Selection is sequence-level. It reduces $\bar{A}_{l,h,p}$ to one score per position and keeps one set of K positions for every head and layer, which is what lets the cache compact. No step needs a forward pass.

One score per position. The position score is the mean of $\bar{A}_{l,h,p}$ over layers and heads. A 7-position max-pool then replaces each score by the largest in its neighbourhood, SnapKV's clustering step [4], so a run of related tokens survives together instead of one spike.

Per-prompt split. The fraction of attention mass outside the recent window, α_a, sets how many of the K cells are distant anchors: $n_{\text{anchor}} = \text{round}(\alpha_a(K - n_{\text{sink}}))$, with α_a floored at 0.70 and capped at 0.95. The rest is the recent window. Retrieval prompts, whose mass sits in the prompt, get more anchors; chat prompts get more window. On the 9737-token prompt $\alpha_a = 0.71$, so K=1024 is 4 sinks, 721 anchors and 299 recent cells.

Keep set. The top n_{anchor} positions by pooled score survive as anchors, plus four sink tokens [2]. Everything else in the prompt is evicted. During decode the keep set stays frozen while the recent window slides.

A per-head gate, removed. Earlier versions sized a per-head budget from each head's peak attention through a closed-form ramp, $K_h = \text{round}(K\,\mu(m_h))$ with $\mu \in [0.7, 1.3]$, and unioned the per-head top-K_h sets into a candidate pool ahead of the steps above. Measured on every cell, that pool holds 96.5 to 99.8% of the prompt

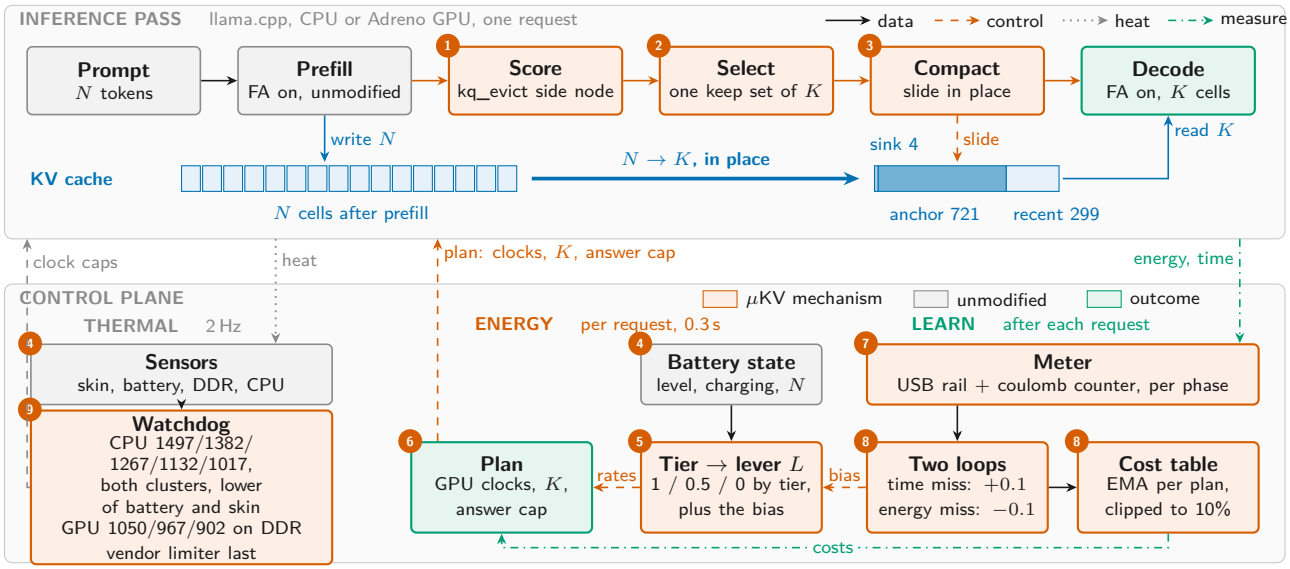

Figure 1: μKV on the phone. Top: the inference pass, with scoring, selection and compaction (1 to 3) as the only changes. Bottom: the thermal, energy and learning columns of the control plane (4 to 9).

at K=1024. The union of 32 per-head sets is the whole prompt, the same effect that stops per-head evictors from compacting. We verified the removal on the 9737-token prompt: with the gate bypassed the keep set is identical, 721 of 721 positions, and so is the generated text. Every cell in this paper ran with the gate in place, which changes nothing; the released configuration bypasses it.

Sizing K. Prompt length under-determines the budget. A short instruction that produces a long answer needs a cache that grows with the output. A long prompt with a short answer needs only a slice. In a controlled retention test on Llama-3.2-1B, the smallest K that still retrieves a fact placed before a span of generated text scales with the span: 256 cells at 64 tokens, 4096 at 2048. The prompt was fixed at 36 tokens. μKV therefore sizes K from the request's demand, the larger of a retrieval floor and a generation floor. The tables below use the nominal K=1024.

3.3 In-place compaction and decode

Compaction through the state API restores survivors into a second context. Peak cache memory doubles at that instant. On unified memory that is fatal: Phi-3-mini at 16K needs 2 × 6144 MiB plus 2.4 GB of weights, and Android killed the process and six co-resident apps, twice. `endurkv_compact_seq()` instead slides survivors into a dense prefix in bounded chunks inside the tensors prefill already allocated. Peak memory does not rise, and the same configuration runs at 878 cells. The two modes produce identical keep sets; across four model families and a 64K sweep their perplexities agree to 0.15%. An earlier variant ran prefill FA-off and swapped the compacted cache into an FA-on context. It reaches the same budget, 733 cells against 721, for 11% more prefill time, and appears in the CPU table as μKV (state-swap).

Decode runs over the K-cell cache with the fused kernel. The keep set stays frozen while the recent window slides. A re-eviction fires when the cache's position span passes 1.25 times the prompt plus the window. On the 4096-token decode it fires once, at 3529 live cells, and the decode-mean live cache is 1980. On the CPU the K half of the cache is stored as q8_0, which halves K-side bytes and pays only above about a 1 GB cache. On the GPU every policy runs f16, because quantized KV is numerically broken on this Adreno driver.

4 THERMAL AND ENERGY CONTROL

4.1 Reduce-only clock watchdog

A userspace daemon samples skin, battery and DDR temperature at 2 Hz and drives three caps: one per CPU cluster and one for the GPU. It lowers a cap one step at a time and never raises it above base. Its ladders are anchored at measured vendor triggers, not chosen in theory. On the CPU the vendor deep-throttles both clusters to 883 MHz the instant the battery reaches 50 ℃ (§6.4). The ladder therefore maps battery 47.0/48.0/48.5/49.0/49.5 ℃ and skin 50.0/51.0/51.5/52.0/52.5 ℃ to the same five caps, 1497/1382/1267/1132/1017 MHz. The two sensors reach that staircase independently and each cluster takes the lower of them, so whichever heats first governs. The floor stays above the cliff, so the watchdog never self-throttles, and DDR above 71 ℃ floors both clusters to 1267 as a backstop. On the GPU the vendor limiter drops to 726 MHz when DDR passes about 64 ℃. A second ladder writes 1050, 967 and 902 MHz at DDR 60, 62 and 63.5 ℃, with 1.5 ℃ hysteresis. The kernel applies the lower of the user cap and the vendor cap, so the ladders act earlier than the vendor and never later. On a cold phone both stay dormant. The watchdog acts on the clock only; §6.4 shows that a cache budget tied to temperature does not cap peak heat.

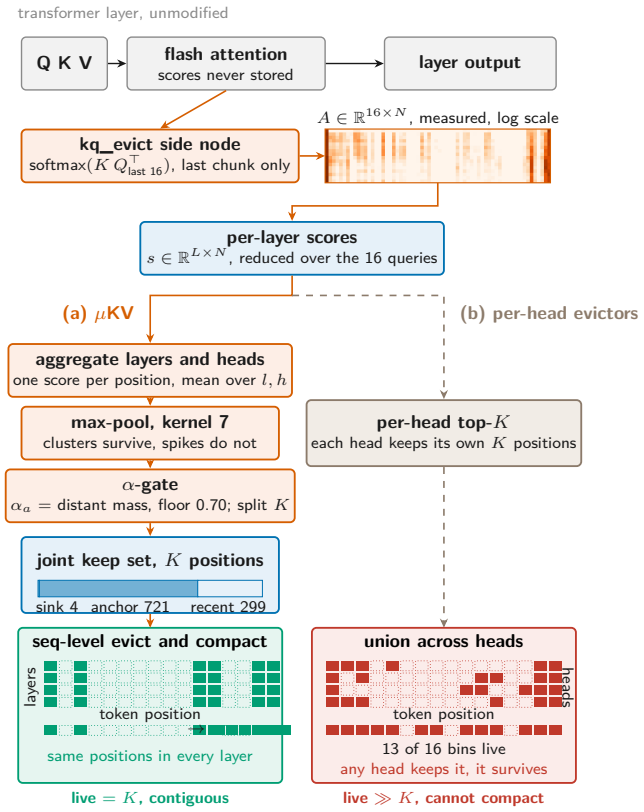

Figure 2: Scores, selection and the two outcomes. Heatmap and keep sets are measured on Llama-3.2-1B, layer 8, 16 position bins: the union over 32 heads keeps 13 bins, the joint set 6.

4.2 Energy-aware scheduling: one lever, two loops

The scheduler runs once per request in about 0.3 s (Figure 3). It reads the battery level, the charging state, the prompt length, and whether the caller fixed an answer length. A tier follows: mains or above 50%, 21 to 50%, and 20% or below. The tier sets a lever $L \in \{1, 0.5, 0\}$ plus a learned bias. L fixes every rate the scheduler trades at. The exchange rate a step must beat is 1.5, 0.8 or 0.5 J saved per second given up. A quality floor and a time slack follow it. So does the answer cap when the caller left the length open: 4096, 1024 or 512 tokens.

Plan. A measured cost table holds energy and time per plan. A plan is a GPU clock cap for prefill, one for decode, and the cache budget. The walk down the plan ladder takes a step only if the predicted saving beats the exchange rate and the predicted time stays inside the budget. The split plan, prefill at 1200 MHz and decode at 902, is the two-configuration schedule of Kim, Imes and Hoffmann [26]. It is also the best measured point by energy-delay product (§6.5).

Backend, and what happens when a model cannot use the GPU.. The table holds CPU plans beside the GPU ones. For the same request they lose on both axes, 822 J and 304 s against 782 and 142 at the top GPU plan, so the walk never chooses the CPU while the GPU is usable. The CPU is the fallback. The engine may lack the Vulkan library, or the caller may force it. A model may also run on the GPU and produce nothing usable. PrismML's 1-bit Bonsai-8B is that case: its Adreno kernels return non-finite logits, and greedy decoding then emits tokens at full speed. Every throughput and thermal metric looks healthy while the text is corrupt. The scheduler therefore grades each run's output, marks a model that fails as GPU-incapable, and re-runs the request on the CPU. On that path the clock is not a lever and the cache does not move, since K=512 scores 0.63 on the worst task and every tier's quality floor rejects it. The answer cap carries the tiering alone. On Bonsai-8B's own measured per-token costs the three caps cost 7853, 5310 and 4886 J.

The cost table is per model. Energy and time per token differ by an order of magnitude between models. Bonsai-8B on the CPU costs 5.1 times the energy that the Llama-3.2-1B row predicts. One shared table would mispredict every other model, and learning from those requests would corrupt the row that was right. Each model therefore keeps its own table, seeded from the measured one. While a table is still a seed the loops report their error but hold the lever, because that error belongs to the seed and not to the phone.

Two loops on model error. After the request the meter reports energy and time per phase. If time ran more than 5% over the budget the plan was chosen under, the performance loop moves the bias 0.1 toward performance. If energy ran more than 5% over its prediction, the energy loop moves it 0.1 toward energy. When both are met the bias decays by 0.05 toward the tier default. The bias is clamped to ±0.3. This is the JouleGuard coupling [25]: the loops act on the model's error, so a disturbance moves the lever and a quiet request moves it back. Knobs only one loop feels belong to that loop: the decode clock and the answer cap to the energy loop, the CPU clock to the watchdog. Only the prefill clock is shared, and the lever arbitrates it. The cost table learns by an exponential moving average clipped to 10% per request, so interference cannot pass for a plan's cost. There is no online policy learning on the user's phone. The decision is a table walk, the table is seeded from measurements, and the loops correct it. We tested whether learning is needed at all. A contextual bandit with the same three contexts, the same reward and one pull per real request explored the GPU clock arms for 24 requests (Table 1). It reproduced the rule at the healthy and low tiers. At the mid tier it preferred a flat 902 MHz, which scores 0.041 higher on that tier's own objective because it saves 21% of the energy for 26% more time. The rule refuses that plan, not because it ranks it lower but because 181 s misses the mid tier's 173 s time budget. So the one place learning helps is a place where the rule's time guard, not its ranking, is what binds. Learning cost 15.7 kJ and 149 minutes of phone time, and 93% of its regret fell in the mandatory warm start. The rule pays none of that, because its table is measured once, offline.

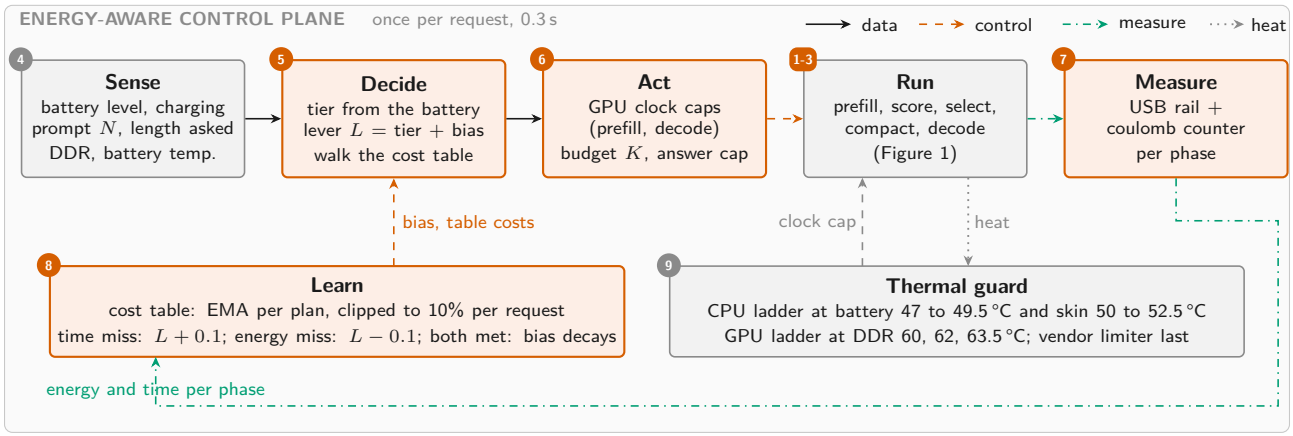

Figure 3: The energy-aware control plane, once per request. Circled numbers are the components of Figure 1, so the same part carries the same number in both figures. Run is that figure's steps 1 to 3. A number appears on two boxes where one stage spans two components.

Table 1: Contextual bandit against the rule, 24 real requests on the phone GPU with Llama-3.2-1B, 9737-token prompt and 1024 output, cooled per pull. Utility is the tier's own objective, so both are scored on one scale. Full per-arm results in the artifact.

Tier	Rule's plan	Util.	Bandit	Util.	Gain
healthy	1200, decode 902	+0.099	1200	+0.100	+0.001
mid	1200, decode 726	−0.285	902	−0.244	+0.041
low	902	−0.455	902	−0.455	0
Cost of learning: 24 pulls, 15.7 kJ, 149 min; 93% of regret in the warm start					

5 IMPLEMENTATION

μKV is a `llama.cpp` extension we call `eviction_bench`: a 4.2K-line driver plus engine patches. The patches add the `kq_evict` side node on the CPU and Vulkan backends, the score reduction and split, `endurkv_compact_seq()`, and a cell-index removal call for multimodal caches. The driver also carries faithful ports of SnapKV, Ada-KV, H2O, TOVA and StreamingLLM in their published configurations. The watchdog and the scheduler are shell daemons on the phone. The cost table is a text file, and a decision is a table walk. Sensors are read from sysfs at 2 to 5 Hz. Energy integrates the USB rail ($V \cdot I$) and the battery coulomb counter, split at the prefill and decode boundary, with charging disabled during every timed cell. The GPU cap writes both the kgsl power-level index and the clock, because the vendor engine clears a plain clock cap mid-request.

6 EVALUATION

Setup. OnePlus 15: Snapdragon 8 Elite Gen 5, 16 GB unified memory, Adreno 840. Models: Llama-3.2-1B, Phi-3-mini-128k and gemma-2-2b-it in Q4_K_M, and PrismML's 1-bit Bonsai-8B [24]. Every timed cell starts behind a cooling gate (DDR $\leq 35\,C$, battery $\leq 33\,C$) with charging off, on one build, under the vendor's own DVFS and thermal engine. The vendor's clock cap is the same for every policy. Baselines run their own papers' selection rules. The tables use a matched budget of 1024. SnapKV and Ada-KV keep 1024 cells per head (window 64, kernel 5; Ada-KV safeguard 0.2). H2O keeps 1024 per head, split evenly between heavy hitters and recent tokens. TOVA keeps 1024 per layer with 4 sinks. StreamingLLM keeps 4 sinks and 1020 recent cells. A second campaign runs every baseline at its own published budget (§6.1). An RTX 4500 Ada carries the four-model LongBench sweep, and a Jetson Orin NX the disjoint-slice perplexity, which needs a second context beside the prompt. μKV runs with K=1024 and the watchdog throughout.

6.1 Sustained decode on the CPU

Table 2 runs a 9.7K-token prompt followed by 4096 generated tokens on the CPU. A cell sustains pressure for 7 to 85 minutes. Figure 4 shows the mechanism. Every policy builds the full prompt cache during prefill. μKV then compacts to 721 cells, and its decode-time re-eviction keeps the cache bounded near 2K. SnapKV can only drop to 5.9K cells, and vanilla grows without bound. On Llama-3.2-1B the bounded cache buys 24.2 against 5.0 tok/s, 2.6 times less wall time and 62% less energy. Its DDR peak is 5 ℃ below vanilla and the joint coolest of any policy, level with TOVA. Both policies run the same vendor-clamped 1.6 GHz, so the heat comes from memory traffic, not the clock.

The realizability gap. SnapKV, Ada-KV, H2O and TOVA keep 3.5K to 6.1K live cells at a nominal budget of 1024, because a cell is freed only when every head and every layer drops it. TOVA selects per layer, and its layers disagree as much as SnapKV's heads do. Their papers report 5 to 10 times smaller caches. On Llama-3.2-1B the reduction is 1.6 to 2.8 times, and on Bonsai-8B, in the same table, 1.0 to 1.6. At their own published budgets the gap widens. We ran 110 LongBench prompts on the phone CPU with Llama-3.2-1B, of which 103 produced output under every policy. SnapKV at 2048 per head keeps 94% of the cache, TOVA at 2048 keeps 77%, Ada-KV at 2048 keeps 71%, and H2O at 20% of the prompt keeps 92%. On the 9737-token WikiText prompt H2O at 20% keeps 99.8%. StreamingLLM at its 2004 keeps 33% and μKV at 1024 keeps 14%.

Table 2: Sustained decode on the CPU, cold start, charging off. One text, retokenized per model. Upper blocks: 4096 generated at K=1024. Lower blocks: 2048 at each policy's own budget. Cells live after prefill.

Model	Policy	Budget	tok/s	Prefill (s)	Wall (s)	Live cells	RSS (MB)	DDR (°C)	Energy (mWh)
Llama-3.2-1B	vanilla (full cache)	1024	5.0	229	1055	9737	2102	59.8	1033
	μKV	1024	**24.2**	**226**	**406**	721 (1980)	2154	**54.4**	**391**
	μKV (state-swap)	1024	22.7	250	438	733 (1984)	2235	58.3	546
	SnapKV	1024	6.8	287	902	5929	2236	56.3	818
	Ada-KV	1024	6.7	252	875	3484	2236	57.1	1002
	StreamingLLM	1024	6.6	250	877	777	2236	56.7	1052
	H2O	1024	5.8	292	1006	6110	2312	54.8	1094
	TOVA	1024	6.0	253	947	4091	2235	54.4	1069
Bonsai-8B (1-bit)	vanilla (full cache)	1024	1.0	1058	5137	10074	4393	**72.2**	5758
	μKV	1024	**4.5**	1073	**2012**	789 (1993)	4517	**62.5**	**2312**
	SnapKV	1024	1.1	1103	4713	10015	4475	67.9	5476
	Ada-KV	1024	1.6	1191	3718	6514	4580	65.2	3971
	StreamingLLM	1024	1.6	1161	3777	858	4580	63.7	4012
	H2O	1024	1.5	1214	3900	9587	4748	63.3	4136
	TOVA	1024	1.6	1149	3841	7116	4580	67.2	4581
Phi-3-mini	vanilla (full cache)	1024	1.2	381	2063	7542	10679	60.6	2046
	μKV	1024	**5.5**	471	**841**	929	10764	**56.3**	**926**
	SnapKV	1024/head	1.7	759	1980	7130	10906	66.0	2530
	Ada-KV	2048	1.7	702	1908	7086	10906	66.8	2479
	StreamingLLM	4+2000	2.0	472	1493	2000	10680	57.1	1311
	H2O	20%	1.7	1092	2305	7542	11018	62.5	2660
	TOVA	2048	1.7	670	1856	6425	10645	62.9	2380
gemma-2-2b-it	vanilla (full cache)	1024	3.9	206	735	6382	4971	58.7	974
	μKV	1024	**9.3**	206	**427**	804	4978	**58.7**	**589**
	SnapKV	1024/head	4.9	496	913	6297	5040	60.6	1005
	Ada-KV	2048	5.0	500	909	6222	5040	60.6	989
	StreamingLLM	4+2000	4.9	234	654	2000	4971	57.9	751
	H2O	20%	3.9	493	1017	6382	5118	56.0	1071
	TOVA	2048	3.9	490	1022	6060	5040	56.0	1063

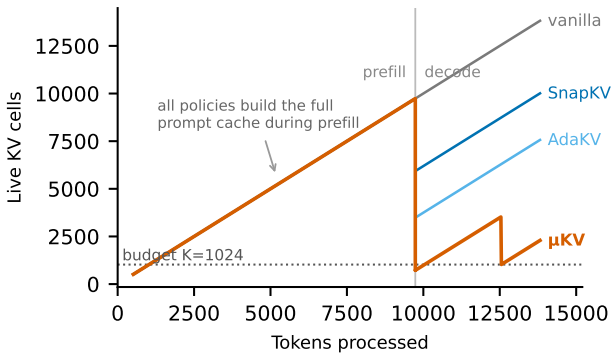

Figure 4: Live KV cells through prefill and decode on Llama-3.2-1B, phone CPU. Every policy builds the full prompt cache. Neither per-head evictor returns to its budget; μKV compacts to 723 cells and its re-eviction holds the cache near 2K.

They also pay the FA-off capture. So they run at 5.8 to 6.8 tok/s and 818 to 1094 mWh against vanilla's 1033, a spread that straddles the full cache rather than beating it. StreamingLLM is the clean control. It compacts to 777 cells, fewer than μKV, yet runs at 6.6 tok/s and 1052 mWh. Retention does not predict speed; the kernel path does. One caveat is ours. This campaign ran StreamingLLM without the fused-kernel flag, and on the GPU, where we re-ran it correctly, it both compacts and stays on the fused kernel and ties μKV (Table 3). The CPU row understates it. That correction is the argument rather than an exception to it: sequence-level selection is what admits both properties at once, and a per-head budget admits neither.

Quality. Self-generation perplexity is about 1.1 for every policy. The quality result is teacher-forced perplexity on a WikiText slice verified disjoint from the prompt (0 of 159 sliding 200-character probes appear in it). Vanilla against μKV scores 8.198 against 8.023 on Llama-3.2-1B, 3.983 against 3.938 on Phi-3-mini, and 6.717 against 6.736 on Bonsai-8B, after evicting about 92% of the prompt cache. Eviction does not cost language-modeling quality. It costs verbatim recall of the spans it drops, which §6.3 probes directly.

Composition with 1-bit weights. Weight compression shrinks the static term of per-token traffic; eviction shrinks the growing one. The two should stack. On Bonsai-8B (Table 2, lower block) μKV gives 4.5 times the throughput, 60% less energy, a live cache of 111 against 1417 MiB, and a DDR peak 9.7 °C cooler, with no change to its mechanism or parameters.

Phi-3-mini and gemma-2-2b. The lower two blocks of Table 2 run the same text on two more models with 2048 generated tokens and every baseline at its published budget. On Phi-3-mini μKV decodes 4.5 times faster than the full cache and uses 55% less energy at 12% of the cells. The per-head and per-layer evictors keep 85 to 100% of the cache at their own budgets and run at 1.7 to 2.0 tok/s. On gemma-2-2b μKV gives 2.4 times the throughput and 40% less energy at 13% of the cells; the per-head and per-layer evictors keep 95 to 100% of the cache, and StreamingLLM 31%.

Table 3: Adreno 840, 4096 generated tokens, f16 KV throughout, cooled per cell. Prompt: 9737 tokens for Llama, 11157 for Phi-3. Median where $n > 1$. Cells retained after prefill. Llama rows group by how a policy selects; the n=3 rows are one interleaved pinned campaign, the n=1 rows the matched-budget campaign, whose own vanilla agrees within 0.5%.

Policy	n	Prefill (s)	tok/s	dec ×	Cells
Llama-3.2-1B, ctx 16384					
vanilla (no eviction)	3	124	24.30	1.00	9741
sequence-level selection					
μKV, K=1024	3	126	**29.02**	**1.19**	723
StreamingLLM, own budget	3	124	29.03	1.19	2005
per-head selection					
SnapKV	1	229	4.76	**0.20**	6592
Ada-KV	1	276	2.86	**0.12**	3519
TOVA	1	176	4.34	**0.18**	4125
H2O	1	279	3.58	**0.15**	6110
controls that isolate one property each					
μKV, compaction off	3	132	25.61	1.05	723
StreamingLLM, forced off the kernel	3	234	5.81	0.24	777
Phi-3-mini, ctx 16384					
vanilla (full cache)	1	471	5.05	1.00	11157
μKV, in place	1	586	**13.36**	**2.64**	878
StreamingLLM, own budget	1	445	10.60	2.10	2000
Ada-KV, gate promoted	1	572	9.13	1.81	4326
SnapKV, gate promoted	1	532	5.80	1.15	10862
H2O, TOVA	*refused by the gate; their CPU rows stand*				

6.2 Sustained decode on the mobile GPU

Table 3 runs the same workload on the Adreno 840 under native GPU DVFS. Two CPU mechanisms are not viable here. The state-swap serializes hundreds of megabytes through host memory, and the Vulkan backend stores FA-off and FA-on caches in different layouts. The state-API defrag pays the same transfer. In-graph scoring removes both needs. Prefill costs 5% more than vanilla on Llama-3.2-1B, and 24% more on Phi-3-mini, where the side node sees a wider model. On Llama-3.2-1B the axis of selection predicts the outcome. μKV decodes at 1.19 times the full cache on 7.4% of the cells, and StreamingLLM at its own documented budget reaches the same 1.19 on 20.6%. StreamingLLM is not our design, so it is the strongest evidence that the mechanism is real rather than a property of our policy. Every per-head evictor instead runs at 0.12 to 0.20 times the full cache with 3.5K to 6.6K cells live. Two controls separate the properties. Disabling compaction leaves μKV's 723 cells but drops it to 1.05, so compaction and not selection carries the throughput. Forcing StreamingLLM off the fused kernel leaves its 777 cells but drops it to 0.24. That forced configuration is how our own first harness ran StreamingLLM, and it was wrong: the reference implementation concatenates survivors, so compaction is intrinsic to it. What separates μKV from StreamingLLM is not speed but what they keep, 723 cells against 2005, and 13 needles against 3. Phi-3-mini shows both of the engine's limits at once. The round trip is killed at 16K and runs only at ctx 8192, where it reaches 1.79 times the full cache. In-place compaction needs one cache and runs at 16K, where μKV decodes at 2.64 times the full cache on 7.9% of the cells. Phi-3 also needs a driver gate. The attention capture on the FlashAttention-off path is numerically broken on this Adreno when the head dimension is neither 64 nor 128, and Phi-3's is 96. Uncorrected runs decode at full speed while emitting

Table 4: Needle retrieval on the phone, 392 cells: hits out of 14 depths per model, 7 at 4K and 7 at 8K. Cells attended: mean live cache after prefill. A quality axis only.

Policy	Phi-3	Llama-1B	Gemma-2B	Bonsai-8B	Cells attended
vanilla (full cache)	14	13	11	14	4858
μKV	**14**	**13**	**11**	**14**	**774**
SnapKV	14	12	11	14	4237
Ada-KV	14	12	11	14	3361
TOVA	14	12	11	14	3670
H2O	14	8	10	13	4212
StreamingLLM	**3**	**3**	**2**	**3**	890

18 to 44% corrupted tokens, so throughput alone never reveals the fault. The gate promotes SnapKV and Ada-KV to the in-graph side node, which is plumbing rather than a policy change, since both score once at the end of prefill. It refuses H2O and TOVA, which re-score every step and would otherwise evict nothing. Under the gate those two promoted evictors reach 1.15 and 1.81 times the full cache while holding 97% and 39% of it, so the realizability gap survives the correction. Quantized KV is numerically invalid on this driver, so every GPU row is f16. The 1-bit Bonsai kernels return non-finite logits on Adreno at full speed, so its phone results are CPU results. We now require an output-validity check before recording any number from a new backend.

Is this one engine? The coupling lives in the memory layout, not in our code. `llama.cpp` holds one contiguous cell array per layer. Paged engines are no different where it matters: a vLLM block has shape [blocks, kv heads, head size, block size] [18], so one block carries every head for a span of positions and is returned only when all of them are finished with it. MLC-LLM [19] pages the same way, and ExecuTorch and the vendor SDKs use static position-indexed tensors. In each of them a per-head budget frees memory only where every head agrees. KV-Compress [17] reports the same limitation independently, stating that per-head eviction "adds fragmentation and cannot realize the theoretical compression rates in physical memory," and fixes it on the server by paging per head inside PagedAttention. That fix needs a per-head block table and a custom kernel. A phone runtime has neither, and a mobile GPU driver makes both expensive. The gap is a property of how KV memory is laid out on devices, not of one engine.

6.3 Retrieval and long-context quality

Needle in a haystack. A fact is buried at one of seven depths in 4K or 8K tokens of filler, and the model is asked for it. The 392 cells cover seven policies, four models, seven depths and two contexts, all behind the cooling gate. Table 4 gives the hits. μKV matches the full cache on every model while attending to 774 cells of its 1024 budget. The score-reading evictors need 3361 to 4237, which is 69 to 87% of what the full cache holds, so the realizability gap appears on retrieval too. StreamingLLM fails retrieval everywhere; its window evicts exactly the middle of the context. Where every policy misses a depth, on Gemma-2B and Llama-1B at 8K, the full cache misses it too, so the model and not the evictor is the cause. We read nothing about speed or energy from this table. The workload generates 64 tokens, so it is prefill-dominated and the two axes are a tie by construction; they are the 4096-token workload of Table 2.

The watchdog never fired on these four-minute cells, so the table is the cache lever alone.

LongBench. Table 5 scores five LongBench tasks on 50 samples each. μKV keeps 12 to 13% of the prompt cache. It stays within 0.1 F1 of the full cache on Llama-3.2-1B and Phi-3-mini, and within 2.8 and 3.7 on gemma-2-2b and Bonsai-8B, where the loss sits in the multi-hop tasks. The per-head evictors land between 1.8 F1 below the full cache and 3.0 above it, at 47 to 96% of the cache. They stay close to the full cache because they nearly are the full cache. StreamingLLM at its own published budget of 2004 cells holds 2.4 times more cache than μKV and scores 4 to 7 F1 lower.

LongBench on the phone. The table above is scored on a desktop GPU because the sweep is 7000 cells: four models, seven policies, five tasks and 50 samples each. A phone pass of that size would take weeks. We therefore ran one model on the device and let it check the ordering. Table 6 scores 103 prompts on the phone CPU with Llama-3.2-1B, every policy at its own published budget rather than a matched one. The result is the desktop result: μKV matches the full cache, at 0.16 F1 above it, while holding 14% of the cache. SnapKV, Ada-KV and TOVA gain 0.6 to 1.1 F1 and hold 71 to 94%. StreamingLLM loses 1.9 F1 at 33%. What separates the policies is the cache each one frees, not the budget it asks for.

6.4 Sustained heat and the watchdog

Figure 5 shows the workload that reaches the cliff. The full cache on Bonsai-8B runs 85.6 minutes and drives the battery to 50.2 ℃. At 50 ℃ the vendor deep-throttles both clusters, the prime cores to 883 MHz, in a sawtooth of 17 dips. The longest lasts 123 s; the run spends 923 s at 883 MHz in total. μKV finishes in 33.2 minutes at a 44.2 ℃ battery peak, before the trigger. The watchdog belongs to μKV and is never given to a baseline in any comparison in this paper. The orange trace is an ablation, included because μKV finishes before the cliff and so cannot show the clock lever by itself. The glide, driven by the battery and the skin ladders, flattens the battery rise from 0.48 to 0.05 ℃/min and settles it at 49.1 ℃. The prime cores never reach 883 MHz and spend 9 s in total below 1 GHz. This decode is bandwidth-bound, so preventing the throttle adds no speed: 1.0 tok/s either way. The watchdog buys thermal sustainability; μKV buys speed and efficiency through the cache. The DDR panel separates the two levers. DDR peaks near 70 ℃ in both full-cache runs, set by memory traffic, while each 883 MHz dip sheds about 8 ℃.

The GPU ladder, measured. On the 4096-token GPU request at 1200 MHz the vendor limiter fires in every cell. Three cooled cells per arm give 1289 against 1390 J, 216 against 214 s, and a DDR peak of 75 against 90 ℃. The ladder saves 7% of energy and 15 degrees for 1% more time.

Cache size is not a temperature actuator. We also tied the cache budget itself to DDR temperature: a four-tier ladder over $K \in \{256, 512, 768, 1024\}$ in symmetric and ratcheted variants, under cool and co-loaded regimes. The mechanism works and the result is negative. Halving K under co-load left the peak DDR unchanged. On a cool phone a smaller cache runs hotter at peak, because the saved bandwidth converts to speed and power. Cache size controls throughput, time and energy. Frequency caps heat. The shipped design keeps the two actuators separate.

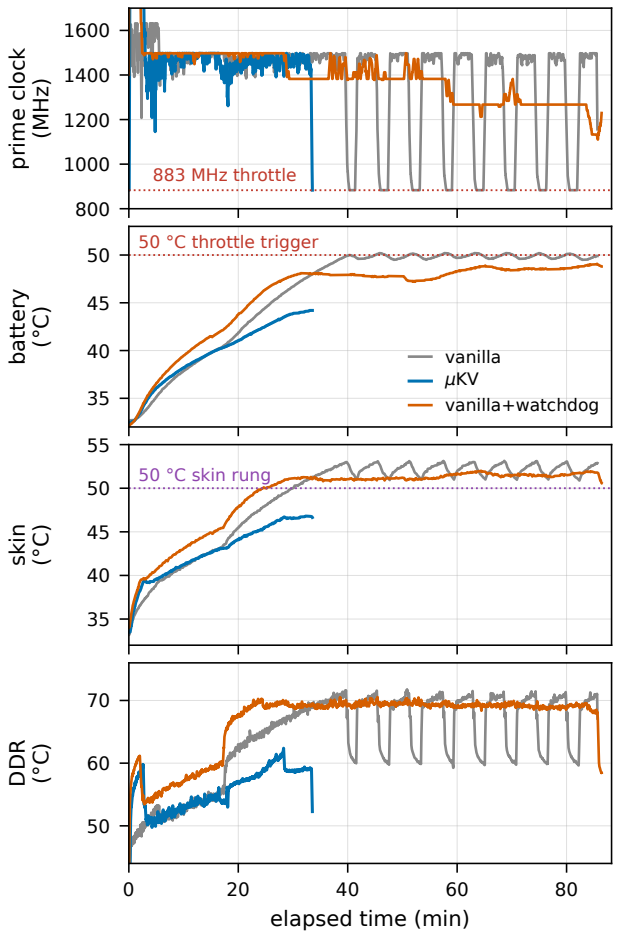

Figure 5: Bonsai-8B on the CPU from a cold start: prime clock, battery, skin and DDR for the full cache, μKV, and the full cache with the watchdog (ablation).

6.5 Energy-aware scheduling

Every number here is from the OnePlus 15 GPU with Llama-3.2-1B and a 9737-token prompt, cooled per cell with charging off.

What each control buys. The GPU clock cap trades prefill energy for prefill time. From 1200 to 902 MHz a 4096-token request falls from 1428 to 1154 J and rises from 221 to 263 s. The next step to 726 MHz buys 3% more energy for 21% more time, so it is never taken. Decode is bandwidth-bound and barely feels the cap. The split plan, prefill at 1200 and decode at 902, keeps the prefill speed and saves 6% of the request for 1.5% of time. The CPU clock is not an energy lever: energy per token is flat within 7% from 883 to 1632 MHz, because power is close to linear in frequency at the pinned voltage. The scheduler leaves it to the watchdog. Below K=1024 the cache saves 3 to 4% of a request while the worst LongBench task loses 37 to 42% of its score, so K stays at 1024. Output

Table 5: LongBench token-F1 on the RTX 4500 Ada, f16 KV for every policy, K=1024; StreamingLLM at its own budget of 2004 cells. Cells: retained after prefill, with the mean of the per-prompt shares beside it.

Model	Policy	HotpotQA	2WikiMQA	MultiFieldQA	Qasper	TriviaQA	Avg	Cells kept
Llama-3.2-1B, ctx 16384								
	vanilla (full cache)	42.14	23.86	45.04	17.55	81.19	41.96	8687 (100%)
	μKV	40.72	25.53	43.23	17.06	83.88	42.08 (+0.1)	826 (13.0%)
	SnapKV	41.14	23.51	45.02	22.81	81.19	42.73 (+0.8)	6717 (82.4%)
	Ada-KV	41.84	22.17	46.16	22.10	81.19	42.70 (+0.7)	3327 (47.3%)
	TOVA	41.61	22.98	45.62	21.93	81.86	42.80 (+0.8)	3824 (53.5%)
	H2O	42.68	24.04	41.87	19.35	81.20	41.83 (−0.1)	6699 (83.1%)
	StreamingLLM	35.61	18.14	36.52	16.51	81.00	37.56 (−4.4)	1994 (23.0%)
gemma-2-2b-it, ctx 8192								
	vanilla (full cache)	46.51	34.51	45.19	36.42	87.67	50.06	6457 (100%)
	μKV	40.32	37.20	39.63	31.71	87.67	47.31 (−2.8)	716 (13.0%)
	SnapKV	40.51	36.81	45.40	38.38	87.44	49.71 (−0.4)	6031 (94.2%)
	Ada-KV	40.51	36.81	44.55	37.26	87.44	49.31 (−0.7)	4064 (66.3%)
	TOVA	41.52	37.31	44.37	37.45	87.44	49.62 (−0.4)	4087 (66.9%)
	H2O	37.38	37.31	43.61	35.37	87.44	48.22 (−1.8)	5869 (92.1%)
	StreamingLLM	41.51	32.67	38.72	28.59	87.67	45.83 (−4.2)	1995 (30.9%)
Phi-3-mini, ctx 16384								
	vanilla (full cache)	46.86	43.51	57.05	36.92	87.87	54.44	9702 (100%)
	μKV	45.97	43.18	55.85	40.80	86.37	54.43 (−0.0)	853 (12.2%)
	SnapKV	46.85	40.01	55.62	38.35	87.20	53.61 (−0.8)	7629 (83.4%)
	Ada-KV	46.90	38.48	56.51	37.33	87.87	53.42 (−1.0)	4056 (50.6%)
	TOVA	54.81	38.67	55.33	50.60	87.87	57.45 (+3.0)	4550 (54.9%)
	H2O	48.32	39.66	56.12	36.97	88.20	53.85 (−0.6)	8702 (91.2%)
	StreamingLLM	40.71	41.88	40.05	30.68	87.87	48.24 (−6.2)	1999 (20.6%)
Bonsai-8B, ctx 16384								
	vanilla (full cache)	35.11	33.39	50.03	38.53	86.15	48.64	8928 (100%)
	μKV	35.16	24.33	48.13	32.30	84.95	44.97 (−3.7)	797 (12.6%)
	SnapKV	38.58	34.30	50.12	38.59	88.15	49.95 (+1.3)	8716 (96.1%)
	Ada-KV	34.46	34.48	50.75	37.56	87.48	48.95 (+0.3)	5293 (68.2%)
	TOVA	37.06	30.19	50.76	37.88	87.48	48.67 (+0.0)	5700 (72.5%)
	H2O	36.66	28.54	49.03	34.97	88.15	47.47 (−1.2)	7798 (90.9%)
	StreamingLLM	34.40	24.97	30.13	30.24	86.26	41.20 (−7.4)	2246 (25.2%)

Table 6: LongBench on the phone CPU, Llama-3.2-1B, 103 prompts scored for every policy, each at its own published budget. Cache: retained after prefill, against the full cache on the same prompt.

Policy	Budget	2Wiki	Hotpot	Qasper	Trivia	Mean	Cache
vanilla (full cache)	full	22.79	29.29	19.96	75.35	36.85	100%
μKV	1024	27.86	25.96	19.01	75.21	**37.01**	**14%**
SnapKV	2048	20.50	31.43	20.72	79.19	37.96	94%
Ada-KV	2048	20.75	31.43	19.66	79.19	37.76	71%
TOVA	2048	18.75	31.37	20.33	79.19	37.41	77%
H2O	20% of N	14.61	31.32	19.58	79.19	36.17	92%
StreamingLLM	2004	22.59	22.63	18.59	76.14	34.99	33%

length is linear at 0.17 to 0.21 J per token above a fixed prefill cost. The answer cap is therefore the largest lever when the caller left the length open.

The loops, provoked. Thirteen cooled requests carried real disturbances (Figure 6). Four idle cores spinning through a mid-tier request pushed its energy 30% over budget. The energy loop took the lever down twice, the walk reached the 902 cap, and two clean requests decayed the bias back. An external 726 MHz cap written 12 s into three low-tier requests, which is what the vendor limiter does, put them 10 to 12% over their time budget. The performance loop took the lever up three times to its clamp, and the energy loop pulled it back once the cap was gone. The phone added a disturbance of its own: its limiter tripped at DDR 64 ℃ mid-decode, and the loops handled it. The one weakness seen was the table learning interference as a plan's cost, which is why learning is now clipped to 10% per request.

Table 7: The plan at each battery tier and its cost on the cable (n=2 cooled cells) and on the battery (n=15, 21 and 10 of the 123 discharge requests). Llama-3.2-1B, 9737-token prompt.

Battery	GPU plan	Cap	Cable J	Batt. J	Batt. s
mains or above 50%	1200, decode 902	4096	1336	885	280
21 to 50%	1200, decode 726	1024	577	748	210
20% or below	902	512	500	506	177
GPU unavailable	CPU, K=1024	tier	822	·	·

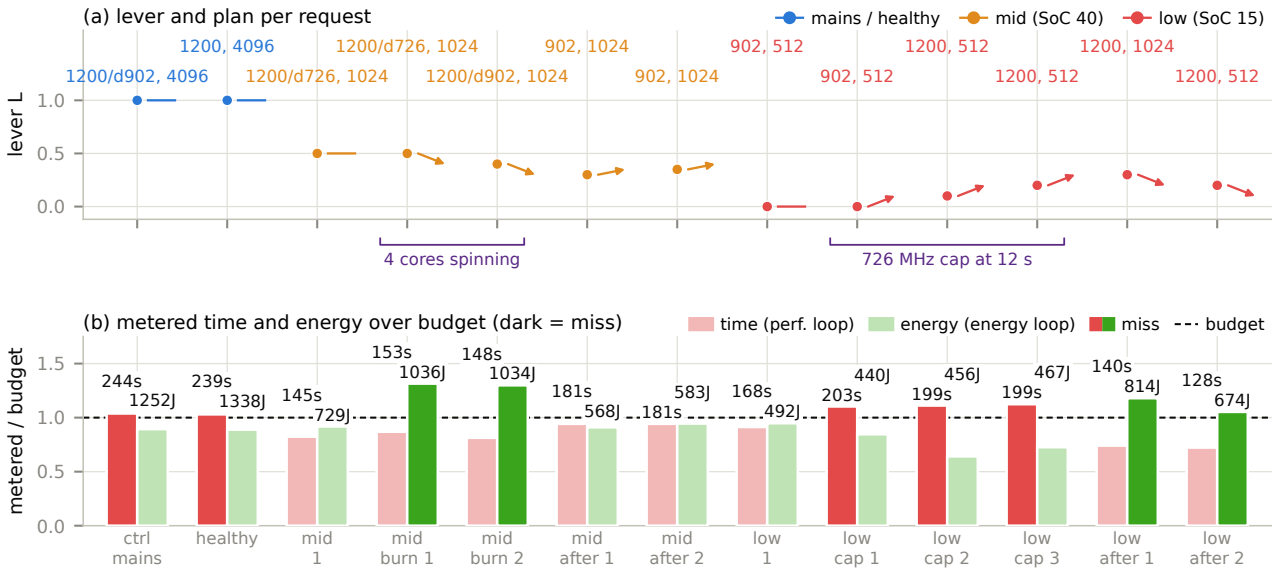

Figure 6: The two loops provoked on the phone: the lever and plan of thirteen cooled requests (top), and their metered time and energy against budget (bottom). Dark bars are misses.

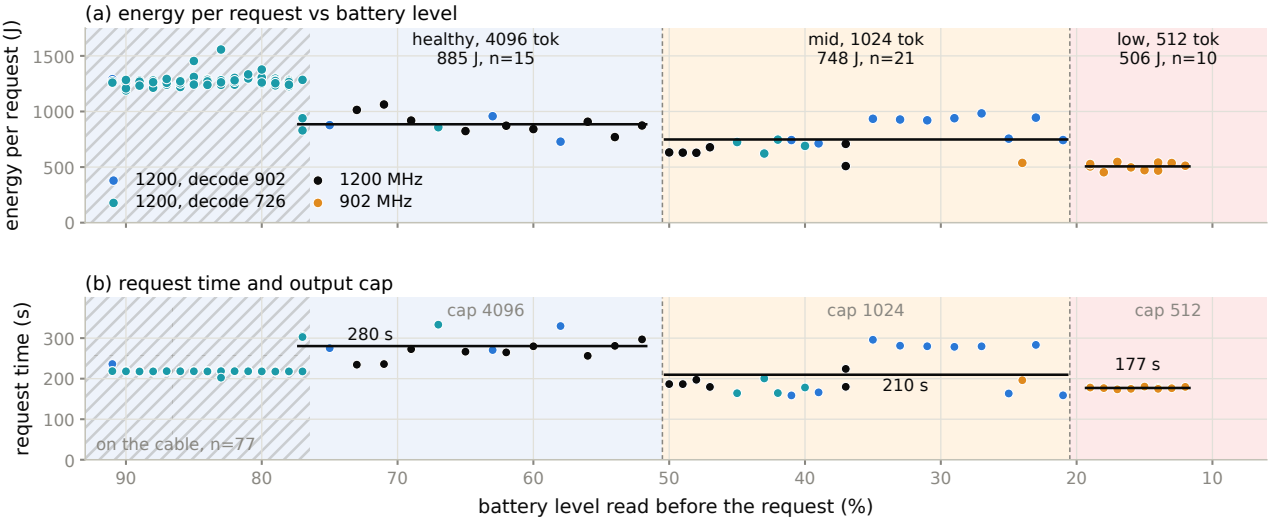

Figure 7: The scheduler through a real discharge: energy and time per request against the battery level it read, with tier bands, the plan chosen and the answer cap. Hatched: cable in.

The real discharge. A loop resident on the phone ran 123 requests through the scheduler from 91 to 11%, with charging disabled and no forced state (Figure 7, Table 7). The scheduler switched on the first request past each threshold: the healthy plan with 4096 tokens down to 52%, the mid plan with the 1024 cap from 50%, the low plan from 19%. On the battery the tiers cost 885, 748 and 506 J per request in 280, 210 and 177 s. That is 15% and 43% less energy than the healthy tier, with time falling 25% and 37%. Two findings came with it. On the battery the phone is a different machine from the phone on the cable. The same healthy request costs 885 J instead of 1336 and takes 280 s instead of 225. The OEM runs the GPU cooler when unplugged, and its limiter caps prefill at 726 MHz within a minute of every request. There the answer cap and the prefill cap carry the saving, and the vendor

pre-empts the decode-clock lever. Per token, healthy and mid cost the same 64 mJ and low 49. Second, a table seeded on the cable mispredicts the battery regime, and the clipped learning tracked it as the run went on. Over the run the mean prediction error was 8.9% on the battery against 3.7% on the cable; the limiter's variable trip time is the difference. PowerBench [28], the only per-rail LLM energy dataset on this phone, and the FUSE study of mobile governors [29] agree with our per-token costs. They also agree that a profiled static setting matches learned governors on a known workload.

6.6 Other results and limitations

Vision-language models. Image tokens enter the same cache, so μKV applies unchanged. On Qwen2.5-VL-3B with one image and a 1024-token answer it cut the live cache from 144 to 23 MiB, 6.3 times smaller, within 1% of full-cache prefill and decode time, with correct answers. Multimodal position encoding gives every token of an image the same position. The position-range cache API cannot express a per-token keep mask there, so we added a cell-index removal call used only for multimodal caches.

Scope. All measurements are on one OnePlus 15. The ladder thresholds are device-specific; the calibration protocol, anchoring at the measured vendor trigger, generalizes. Energy is total-device energy from the USB rail and the coulomb counter, not a die-attached meter. Absolute joules hold within about 10%; orderings hold. The controller needs privileged telemetry, available on rooted Android. On a stock phone the Android 16 thermal-headroom and ADPF hints are the no-root path to the same decisions. DDR frequency and NPU prefill are levers we have not measured. Eviction still costs verbatim recall of the spans it drops, even though prediction and retrieval are preserved.

7 RELATED WORK

KV-cache eviction. is surveyed in §2. Per-head budget allocation is Ada-KV's idea [6]; we credit it and do not reprove its bound. CriticalKV [7] and DefensiveKV [8] refine the score and the worst case. AhaKV [9] corrects the per-token score under a uniform budget. R-KV [22] and DynamicKV [23] argue for task-aware budgets, as our demand-based sizing does. None reports a temperature, a watt, or a resident set on a phone.

Mobile inference systems. Table 8 lists what mobile KV-cache work has measured. PowerInfer-2 [12] and LLM in a Flash [13] optimize weight access. KVSwap swaps the cache to disk on a Jetson. DynaKV offloads to UFS on the CPU. PerCache [16] shortcuts whole inference passes at the application level and composes with μKV, which bounds the cells attended when PerCache misses. MLC-LLM [19], ExecuTorch, MediaPipe and the vendor SDKs use the unmodified cache. Weight compression, from 4-bit K-quants to 1-bit Bonsai [24], attacks the other term of per-token traffic and composes with μKV (§6.1).

Energy and thermal control. zTT [20] and FUSE [21] govern frequency from temperature or phase, with no cache or battery signal. The FUSE study of governors under LLM inference [29] finds that a profiled static setting matches learned governors, which is why our scheduler walks a measured table instead of learning online. PowerBench [28] measures per-rail LLM energy on this phone. EnerInfer [27] is energy-aware on device but excludes battery level. The two-configuration theorem [26] and JouleGuard [25] supply the plan shape and the loop coupling. We know of no prior system that reads attention-internal state and battery and thermal sensors in one decode loop on a phone.

8 CONCLUSION

μKV makes attention-driven KV eviction work on a phone by respecting the engine. Scores come from inside the FlashAttention graph. One keep set is what the shared cell array can free. Compaction never needs a second context. Around it, a reduce-only clock watchdog holds the CPU below the vendor's throttle, and a per-request scheduler with one lever and two loops spends the battery according to its level. On a OnePlus 15 the result is 4.8 times the full cache's decode throughput on the CPU, at 7% of the cells and 62% less energy, and 1.23 times on the GPU. Retrieval matches the full cache and LongBench nearly does. A 1-bit model finishes where the full cache throttles. A real discharge's lower tiers cost 15% and 43% less per request. The negative results matter as much. The cache is the phone's efficiency actuator and the clock its temperature actuator. Every per-head budget that looks good on a server is a full cache in disguise on the device.

REFERENCES

[1] Z. Zhang et al. "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of LLMs." NeurIPS, 2023.
[2] G. Xiao et al. "Efficient Streaming Language Models with Attention Sinks." ICLR, 2024.
[3] M. Oren et al. "Transformers are Multi-State RNNs." arXiv:2401.06104, 2024.
[4] Y. Li et al. "SnapKV: LLM Knows What You're Looking For Before Generation." NeurIPS, 2024.
[5] Z. Liu et al. "KIVI: Tuning-Free Asymmetric 2bit Quantization for KV Cache." arXiv:2402.02750, 2024.
[6] Y. Feng et al. "Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation." NeurIPS, 2025.
[7] Y. Feng et al. "CriticalKV: A Perturbation Perspective on KV Cache Eviction." ICML, 2026.
[8] Y. Feng et al. "DefensiveKV: Worst-Case Risk Control for KV Cache Eviction." ICLR, 2026.
[9] Y. Gu et al. "AhaKV: Adaptive Holistic Attention-Driven KV Cache Eviction." arXiv:2506.03762, 2025.
[10] T. Dao et al. "FlashAttention: Fast and Memory-Efficient Exact Attention." NeurIPS, 2022.
[11] T. Dao. "FlashAttention-2: Faster Attention with Better Parallelism." arXiv:2307.08691, 2023.
[12] Y. Song et al. "PowerInfer-2: Fast LLM Inference on a Smartphone." arXiv:2406.06282, 2024.
[13] K. Alizadeh et al. "LLM in a Flash: Efficient Inference with Limited Memory." arXiv:2312.11514, 2023.
[14] G. Gerganov. "llama.cpp." https://github.com/ggerganov/llama.cpp.
[15] J.-H. Kim et al. "KVzip: Query-Agnostic KV Cache Compression with Context Reconstruction." NeurIPS, 2025.
[16] K. Liu et al. "PerCache: Predictive Hierarchical Cache for RAG Applications on Mobile Devices." arXiv:2601.11553, 2025.
[17] I. Rehg. "KV-Compress: Paged KV-Cache Compression with Variable Compression Rates per Attention Head." arXiv:2410.00161, 2024.

Table 8: Mobile KV-cache work by what each paper measured, verified from its evaluation section. Eviction has not been deployed end to end on a phone GPU before.

System	Venue	Hardware evaluated on	Phone	Compute used	KV mechanism
KV eviction, but never on a phone					
SnapKV, Ada-KV, H2O, TOVA, PyramidKV	2023 to 2024	A100-class servers	·	server GPU	eviction
KVSwap	MobiSys 2025	Jetson Orin AGX 64 GB and NVMe	·	board GPU	offload to disk
MobiLoRA	ACL 2025	NVIDIA AGX Orin	·	board GPU	cache reuse
KeyDiff	NeurIPS 2025	A100; unnamed Samsung phone (kernel only)	•	not stated	eviction
On a real phone, but never eviction, and never the GPU					
DynaKV	2025	OnePlus Ace5 Pro, 12, Ace3, Ace2	•	CPU only	offload to UFS
ActiveFlow	2025	OnePlus 12, Pixel 6, Infinix ZERO 30	•	CPU only	weights, not KV
PowerInfer-2	MobiSys 2025	OnePlus 12, OnePlus Ace 2	•	CPU and NPU	neurons, not KV
PerCache	2026	Pixel 7, Redmi K60 Pro, S22 Ultra, Ace 6	•	not stated	QKV reuse (RAG)
On a phone GPU, but no cache management					
LLMs in Your Pockets	2026	7 devices: Adreno 750/730, Mali G720/G78	•	CPU, GPU, NPU	none
LLM Inference at the Edge	2026	S24 Ultra (Adreno 750), iPhone 16 Pro	•	GPU	none
μKV (this work)		**OnePlus 15, Adreno 840**	•	**CPU and GPU**	**eviction and compaction**

[18] W. Kwon et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention." SOSP, 2023.

[19] MLC Team. "MLC-LLM: Universal LLM Deployment." https://github.com/mlc-ai/mlc-llm.

[20] S. Kim et al. "zTT: Learning-Based DVFS with Zero Thermal Throttling for Mobile Devices." MobiSys, 2021.

[21] S. Liu et al. "FUSE: Fine-Grained Power Management for Mobile Devices." SenSys, 2022.

[22] X. Cai et al. "R-KV: Redundancy-aware KV Cache Compression for Reasoning Models." arXiv:2505.24133, 2025.

[23] X. Zhou et al. "DynamicKV: Task-Aware Adaptive KV Cache Compression for Long Context LLMs." arXiv:2412.14838, 2024.

[24] PrismML. "Bonsai: Commercially Viable 1-bit LLMs." 2026. https://prismml.com/news/bonsai-8b.

[25] H. Hoffmann. "JouleGuard: Energy Guarantees for Approximate Applications." SOSP, 2015.

[26] D. H. K. Kim, C. Imes, and H. Hoffmann. "Racing and Pacing to Idle: Theoretical and Empirical Analysis of Energy Optimization Heuristics." CPSNA, 2015.

[27] B. Zou et al. "EnerInfer: Energy-Aware On-Device LLM Inference." arXiv:2606.23001, 2026.

[28] G. Cai et al. "Is Your NPU Ready for LLMs? A Power Benchmark of On-Device Inference." arXiv:2607.05475, 2026.

[29] Z. Zhang et al. "Dissecting the Impact of Mobile DVFS Governors on LLM Inference Performance and Energy Efficiency." arXiv:2507.02135, 2025.